

Appunti IEC 61850: classi e modelli concettuali

Versione aggiornata. Le parti originali sono invariate; i riquadri arancioni sono le nuove integrazioni.

Modelli concettuali di IED

Server → Logical Device (LD) → Logical Node (LN) → Data Object (DO) → Data Attribute (DA)

Server

Il server è la vista di rete di un dispositivo fisico e possiede vari attributi:

- ServiceAccessPoint [1..n]
- LogicalDevice [1..n]
- TPAAppAssociation [0..n]: identifica un client con cui il server mantiene un'associazione
- MCAAppAssociation [0..n]: identifica un subscriber che mantiene un'associazione
- File [0..n]: file system del dispositivo

E il metodo GetServerDirectory(), strettamente collegato all'attributo ServiceAccessPoint che identifica il server. Ogni comunicazione con il server avviene tramite ServiceAccessPoint.

Servizi di associazione: Associate (apertura), Abort (chiusura immediata), Release (chiusura).

Un client può esplorare un dispositivo con questa gerarchia di servizi:

GetServerDirectory → GetLogicalDeviceDirectory → GetLogicalNodeDirectory → GetDataDirectory → GetDataDefinition

Il modello Application Association

Definisce il modello di comunicazione tra i dispositivi specificando 2 classi: Client-Server (two party) e Publisher-Subscriber (multicast).

NUOVO / INTEGRAZIONE

Due piani da tenere separati: livello PLC e livello di rete

Function Block (FUNCTION_BLOCK) è un concetto di **IEC 61131-3** (linguaggio del PLC), non di IEC 61850. È una POU (Program Organization Unit) istanziabile, come una classe (supporta anche EXTENDS). Nelle implementazioni server su PLC ogni elemento del modello 61850 è realizzato come FB; il prefisso FB_ è solo una convenzione di nome.

Concetto IEC 61850	Realizzazione nel PLC (esempio)
Logical Node LLN0	FB_LN_IED_LD1_LLN0
Data Object Beh	FB_DO_IED_LD1_LLN0_Beh
DataSet DS1	FB_DS_IED_LD1_LLN0_DS1
Report Control Block	FB_ScsmUrCBImplClass / FB_ScsmBrCBImplClass

In Wireshark si vede solo il livello di rete (LN, DO, DA, FC, MMS): i nomi FB_ non compaiono mai nei pacchetti. Nella relazione conviene distinguere sempre i due piani.

Logical Device (LD)

- LDName: ObjectName, cioè un identificatore relativo ad un livello gerarchico
- LDRef: ObjectReference, identificatore univoco nell'intero sistema
- LogicalNode [3..n]: LogicalNode0 (LN0), LogicalNodePhysicalDevice (LPHD), ed almeno un logical node di tipo funzionale

- Services

L'ObjectReference si costruisce concatenando gli ObjectName di tutto il path.

LogicalNode (LN)

I LogicalNode si dividono in LN di controllo del dispositivo fisico (LPHD), LN di controllo del dispositivo logico (LN0), LN di funzionalità.

Attributi:

- LNName: ObjectName
- LNRef: ObjectReference
- Data [1..n]
- Dataset [0..n]
- Buffered Report Control Block [0..n]
- Unbuffered Report Control Block [0..n]
- LogControlBlock [0..n]

LN0 (in più): SettingGroupControlBlock [0..1], Log [0..1], GooseControlBlock [0..n], GseControlBlock [0..n], MulticastSampledValueControlGroup [0..n], UnicastSampledValueControlGroup [0..n]

Metodi: GetLogicalNodeDirectory(), GetAllDataValue()

NUOVO / INTEGRAZIONE

LLN0, LPHD e LN funzionali: perché LLN0 contiene DataSet e RCB

LLN0 rappresenta il Logical Device nel suo insieme, **LPHD** rappresenta il dispositivo fisico, gli altri LN (XCBR interruttore, MMXU misure, ...) rappresentano funzioni applicative. Per questo i DataSet, i Report Control Block, i GOOSE control block e i Log stanno in LLN0: sono risorse condivise dal dispositivo e non legate a una singola funzione.

Data Object tipici di LLN0 e relativa CDC (Common Data Class):

DO	CDC	Significato
NamPlt	LPL	Name Plate: targhetta (descrizione, versione, ...)
Beh	ENS	Behaviour: comportamento effettivo, sola lettura
Health	ENS	Stato di salute del nodo
Mod	ENC	Mode: modo operativo, controllabile dal client

La **CDC** è il "tipo" del Data Object e stabilisce quali Data Attribute contiene: ENS = Enumerated Status (sola lettura), ENC = Enumerated Controlled (accetta comandi).

Nota di implementazione: in Structured Text MOD è un operatore, quindi la variabile si chiama Mod_ e il nome 61850 corretto si forza con sObjectName:='Mod'. Il nome in rete deve restare quello dello standard.

La classe DATA

- DataName: identificatore di istanza, dentro LogicalNode
- DataRef: identificatore globale, path a partire dal Logical Device di appartenenza
- Presence: flag che indica il dato come necessario (mandatory) / opzionale
- DataAttribute [0..n]
- CompositeCDC [0..n]: è di tipo DATA, quindi struttura ricorsiva
- SimpleCDC [0..n]

DataAttribute, CompositeCDC, SimpleCDC formano la struttura dati che specifica l'informazione; non possono essere tutti null, almeno uno di loro deve contenere un valore.

DataAttribute

- DataAttributeType (DAType)
- Functional Constraint (FC): Status Information (ST), Control Information (CO)
- TrgOp [0..n] (trigger option di tipo TriggerCondition)

Il Functional Constraint serve per indicare quali servizi possono essere applicati al DataAttribute.
FCD: insieme di DataAttribute aventi lo stesso FC.

NUOVO / INTEGRAZIONE

Functional Constraint: le famiglie principali

L'FC è una categoria applicata ai **Data Attribute** (non ai nodi) e compare nei riferimenti MMS, ad esempio IEDLD1/LLN0\$BR\$brcb101.

FC	Significato	FC	Significato
ST	Status (stato)	CF	Configurazione
MX	Misure (analogiche)	DC	Descrizione
CO	Controllo	RP	Report Control Block unbuffered
SP	Setpoint	BR	Report Control Block buffered

Il Logical Node "funzionale" (che implementa una funzione di automazione) e il Functional Constraint (categoria dell'attributo) sono due concetti diversi pur usando la stessa parola.

Modelli concettuali delle comunicazioni

Ogni condizione in base alla quale può essere avviato il trasferimento dati è un attributo di TriggerCondition. Ogni motivo per cui un server può decidere di inviare un report è uno dei campi (bit) che compongono il tipo.

Classe TriggerCondition

- DataChange: dchg
- Quality change: qchg
- Data update: dupd
- Integrity: scade un periodo fisso (IntgPd), il server invia comunque tutti i valori
- General interrogation: il client richiede esplicitamente la lettura completa del data set

Un evento parte solo se la condizione è abilitata nel control block e l'attributo la supporta. Ogni dato incluso porta un bit che indica perché è stato incluso. È il campo che serve in Wireshark per capire il motivo di ogni invio.

NUOVO / INTEGRAZIONE

Differenza tra dchg e dupd

- **dchg**: il valore è cambiato.
- **qchg**: è cambiata la qualità (es. valido/non valido).
- **dupd**: il dato è stato riscritto, anche con lo **stesso** valore. Una riscrittura identica genera un report solo dove dupd è abilitato.
- **Integrity** funziona solo se abilitato in TrgOps e con IntgPd > 0; il periodo è in **millisecondi**.

Ogni nodo logico sa le informazioni necessarie per svolgere il proprio compito. Il nodo logico è in grado di verificare che i dati da lui ricevuti siano completi, validi e della qualità giusta. Il criterio più importante per il client è l'età dei dati. Il punto chiave della comunicazione tra dispositivi è la standardizzazione dei dati richiesti o inviati e la definizione degli attributi di validità (qualità).

Il DATA_SET viene trasmesso tramite GSE (GOOSE o GSSE), report, logging. Il DATA_SET è un descrittore che specifica a client e server quale è la struttura dei messaggi che possono essere scambiati, limitando la comunicazione ai soli valori referenziati.

NUOVO / INTEGRAZIONE

Cos'è un DataSet in pratica

È un insieme di riferimenti a dati (FCDA, Functionally Constrained Data Attribute) definito nel proprio blocco (es. FB_DS_...). Serve a raggruppare i valori da inviare insieme in un unico report invece di leggerli uno a uno. Il DataSet e il control block che lo usa devono stare nello stesso Logical Node.

GOOSE (Generic Object Oriented Substation Event)

Il messaggio GOOSE è strutturato da un DATA_SET mentre GSSE è una semplice lista di informazioni. Lo scambio di messaggi è basato sul meccanismo Publisher/Subscriber. Il mittente scrive il messaggio sul suo buffer locale e il ricevente legge il suo buffer locale. L'aggiornamento dei buffer è a carico del sistema di comunicazione, la distribuzione delle informazioni viene gestita da una classe chiamata GSECC (Generic Substation Event Control Class) del sender.

Ogni volta che il valore di almeno uno dei DataAttribute cambia, il buffer del publisher viene aggiornato; il meccanismo di aggiornamento dei buffer riceventi dipende dal protocollo di comunicazione.

Goose Control Block

- GoCBName: identificatore locale al LNO
- GoCBRef: identificatore globale
- GoEna: flag di abilitazione dell'invio messaggio
- ApplId: LogicalDevice di appartenenza
- DataSet: riferimento al DATA_SET contenuto nel messaggio
- ConfRev: numero di aggiornamenti dalla definizione del DATA_SET

Report e Logging

Report e Log vengono costruiti sulla base del DATA_SET che contiene i riferimenti strutturati sugli eventi (DATA) da monitorare: Event Monitor (decide come e quando inviare l'informazione), Report Handler, Log Handler. Report e Log vengono specificati nel modello mentre l'Event Monitor esula dal campo della 61850.

Il control block controlla le modalità di invio delle informazioni provenienti da uno o più Logical Node ad un singolo client. Ogni client è associato ad un singolo Control Block; per trasmettere a più client servono più control block, quindi ad un DATA_SET possono essere assegnati n control block in base alle esigenze dei client.

In fase di progettazione vengono stabiliti i dati che possono essere emessi dalla macchina (IED), si raggruppano i riferimenti a questi dati nel DATA_SET a propria discrezione, e ad ogni DATA_SET viene associato un insieme di Control Block. I client possono richiedere l'invio dei dati richiedendo un control block; da quel momento diventa di sua proprietà. In fase di configurazione vengono create le istanze e le associazioni tra client e control block, poi vengono definite le opzioni che definiscono il contenuto dei report.

Report Control Block: Buffered

Attributi (definiti dal costruttore, sola lettura, identificano l'oggetto in ambito server):

- BRCBName: identificatore locale
- BRCBRef: identificatore globale

- **DatSet:** riferimento al DATA_SET contenuto nel messaggio
- **ConfRev**
- **RptId:** ID report lato client
- **RptEna:** flag di abilitazione/disabilitazione dei report

Il control block e il dataset associato devono appartenere allo stesso logical node.

Dati di configurazione, variabili di controllo per invio report:

- **BufTm:** tempo di bufferizzazione. Se 0, appena viene rilevato un cambiamento interno viene preparato il report; se positivo vengono raccolte tutte le notifiche per la durata dell'intervallo. Se si presenta una nuova notifica su un attributo appena modificato viene inviato un nuovo report.
- **SqNum:** numero di sequenza incrementato a ogni report
- **TrgOp:** trigger condition DATA/DataAttribute per report
- **IntgPd:** tempo di attesa tra report periodici, manda tutto il DATA_SET ignorando le trigger condition
- **GI:** controlla l'invio dei valori nel DATA_SET su richiesta client
- **PurgeBuf:** richiesta svuotamento buffer
- **EntryId:** entry di ogni evento nel buffer
- **TimeOfEntry:** timestamp dell'evento, secondi, frazioni di secondo e qualità
- **OptFlds:** flag da includere o no

Flag di OptFlds: SequenceNumber, ReportTimeStamp (tempo emissione report), ReasonForInclusion (causa invio), DataSetName, DataReference, BufferOverflow, EntryId, ConfRevision.

Il client in configurazione ottiene priorità su un Buffered Report Control Block; in fase di esecuzione, con un flag, può disabilitarlo.

NUOVO / INTEGRAZIONE

OptFlds e TrgOps: cosa significa ogni flag

- **SequenceNumber:** numero progressivo nel report, utile per rilevare report persi.
- **ReportTimeStamp:** ora di generazione del report.
- **ReasonForInclusion:** per ogni dato, il motivo dell'inclusione (dchg, qchg, dupd, integrity, GI).
- **DataSetName:** nome del DataSet nel report.
- **DataReference:** riferimenti dei dati; se FALSE il client deve ricavarli dalla struttura del DataSet.
- **ConfRevision:** invia ConfRev nel report, così il client rileva modifiche di configurazione.
- **BufferOverflow** ed **EntryID:** hanno senso solo nei buffered, per questo negli unbuffered sono sempre FALSE.

Configurazione statica vs runtime: RptID, DataSet, ConfRev, OptFlds, TrgOps, IntgPd sono impostati dal costruttore/progetto. RptEna, Resv, GI, PurgeBuf, EntryID (scritto dal client al resync) sono attributi di **runtime**, scritti dal client. Finché un client non scrive RptEna=TRUE nessun report parte.

Convenzione di progetto (non dello standard): sigle 101/201/301 indicano il DataSet associato (DS1, DS2, DS3), con un unbuffered e un buffered per ciascuno.

Unbuffered Report Control Block

Differenze principali: non possiede buffer e tutti gli attributi legati ad esso, l'invio del report è immediato senza un tempo prestabilito per la raccolta dei dati. BufferedOverflow: false, EntryId: false (obbligatoriamente). Cambiano anche le classi usate e il metodo per la registrazione.

Confronto

Buffered è migliore se non possiamo permetterci perdite di dati (ogni misurazione è significativa per il risultato). **Unbuffered** se il dato più importante per noi è l'ultimo raccolto (real time monitoring).

NUOVO / INTEGRAZIONE

Buffered vs Unbuffered: tabella completa

Aspetto	Unbuffered	Buffered
Eventi con client assente	persi	accodati nel buffer del server
Functional Constraint	RP	BR
Riserva	Resv (booleano)	ResvTms (tempo di riserva)
EntryID	assente	attributo scrivibile dal client
TimeOfEntry / PurgeBuf	assenti	presenti
Classe e registrazione nel PLC	FB_ScsmUrCBImplClass / AddUnbufferedReportControlBlockToContainer	FB_ScsmBrCBImplClass / AddBufferedReportControlBlockToContainer

Cosa cambia nella sola configurazione: a parità di condizioni gli unici parametri diversi sono BufferOverflow ed EntryID, ma non sono ciò che rende buffered il blocco: sono la sua "firma". La vera differenza è il comportamento (buffer degli eventi) e gli attributi di runtime.

Resync con EntryID: alla riconnessione il client scrive nell'attributo EntryID l'ultimo ID ricevuto e il server rinvia solo le entry successive. Il flag EntryID in OptFlds decide solo se l'ID compare nei report inviati; per un resync affidabile va tenuto a TRUE. BufferOverflow segnala che il buffer si è riempito e alcune entry vecchie sono state scartate (buco nello storico).

Nota sul tuo appunto: anche l'unbuffered ha BufTm (tempo di raccolta delle notifiche). Ciò che manca è il buffer degli eventi quando il client non è connesso, non la possibilità di avere un tempo di raccolta. Da verificare sulla IEC 61850-7-2 per la tesi.

Log Control Block

Specifica le modalità di salvataggio dei dati su un supporto permanente. Indica quali valori devono essere salvati tramite il DATA-SET associato e con quale modalità. Modalità buffer circolare: la dimensione dei log viene decisa in fase di configurazione di sistema. I pacchetti vengono salvati in sequenza temporale (FIFO) e c'è un flag che permette l'abilitazione del logging.

Differenza rispetto ai report:

- Nei report il client abilita (RptEna) e riceve i dati subito.
- Nel log LogEna abilita la registrazione interna al dispositivo. Il client non riceve nulla finché non interroga il log con il servizio di lettura, indicando un intervallo di tempo.

Linguaggio SCL

Si basa su XML e UML. SCL usa il termine sottostazione come sinonimo di impianto elettrico. Il compito del modello è mettere in relazione un nodo logico con la sua funzione della sottostazione e ricavare un'identificazione funzionale del nodo logico.

Il concetto fondamentale di LNode in SCL è fungere da ponte logico tra la struttura fisica della sottostazione elettrica e le funzionalità software implementate negli IED. Ogni funzione di automazione viene scomposta in nodi logici che possono essere allocati a uno o più nodi fisici.

Classe LNode

- InInst: identificatore dell'istanza LN

- InClass: identificatore classe LN, riferimento al Logical Node che contiene la funzione di automazione
- iedName: nome dell'IED usato per contenere LN
- IdInst: istanza del LogicalDevice che contiene il LogicalNode
- prefix
- InType: definizione del LogicalNode contenente molte informazioni dettagliate

IdInst + prefix + InClass + InInst identifica il nodo logico del contesto nel quale viene dichiarato il Data Object.

- DoName: nome del Data Object
- DaName: attributo del Data Object del DoName
- FC: Functional Constraint, seleziona un insieme di DataAttribute corrispondenti al constraint

File: .ICD .SSD .SCD

NUOVO / INTEGRAZIONE

Verifiche sperimentali con libIEC61850 e Wireshark (spunti per la tesi)

Filtro Wireshark: mms. Si osservano le Write su RptEna e gli InformationReport inviati dal server.

- L'intervallo tra report di integrità corrisponde a IntgPd (2000 ms per DS1, 5000 ms per DS2/DS3)?
- I campi presenti nel report corrispondono a OptFlds? (es. DataReference FALSE: il report non porta i riferimenti)
- Cambiando un dato, DataChange genera il report atteso? Una riscrittura dello stesso valore lo genera solo dove DataUpdate è TRUE?
- Il campo ReasonForInclusion di ogni dato coincide con la causa reale (dchg, qchg, dupd, integrity, GI)?
- Con un urcb e un brcb sullo stesso DataSet: disconnetti il client, genera variazioni, riconnetti. Il buffered deve rinviare le entry arretrate con EntryID crescenti, l'unbuffered nulla.